

OpsFlow 核心引擎

从模板到执行 — 完整流程详解

2026-06-03

Generated: 2026-06-03 | Source: backend/opsflow/doc/

概述

OpsFlow 从模板到执行的完整链路：

前端画布编辑 → 保存 pipeline_tree → 发布版本 → 创建执行(冻结快照) → 启动(异步派发)
→ Celery Worker 执行 → bamboo-engine DAG 遍历 → PluginService 路由 → BasePlugin 执行
→ post_set_state 信号 → 状态追踪 → WebSocket 推送

第一阶段：模板编辑与发布

1.1 前端保存

用户在 DesignCanvas 上拖拽节点、配置参数后，点击保存：

```
PATCH /api/opsflow/templates/{id}/
{
  "pipeline_tree": {
    "nodes": [
      {"id": "n1", "label": "Shell", "atom_type": "shell",
       "params": {"command": "df -h"}, "node_type": "", "x": 50, "y": 40},
      {"id": "n2", "label": "End", "node_type": "end_event"}
    ],
    "edges": [
      {"from": "n1", "to": "n2"}
    ]
  },
  "target_hosts": ["192.168.1.100"],
  "global_vars": {"env": "prod"}
}
```

文件: [views/template_views.py] → [models/template.py:FlowTemplate]

保存时触发:

1. [cleanup_unused]

1.2 版本发布

用户点击 Publish / New Version:

操作	接口	说明
草稿→发布	<code>`POST /templates/{id}/confirm_draft/`</code>	<code>`is_draft=False` + `publish_snapshot()`</code>
已发布→新版	<code>`POST /templates/{id}/publish/`</code>	<code>`publish_snapshot()`</code>
查看版本	<code>`GET /templates/{id}/versions/`</code>	TemplateVersion 列表
回滚	<code>`POST /templates/{id}/rollback/`</code>	恢复指定版本的 pipeline_tree

[publish_snapshot()] 动作 ([models/template.py:FlowTemplate]):

```
def publish_snapshot(self, user=None, version_note=""):
    if self.version is None:
        self.version = 1
    # 提取子流程引用信息
    subprocess_refs = {}
    tree = self.pipeline_tree or {}
    for node in tree.get('nodes', []):
        if node.get('node_type') == 'subprocess':
            params = node.get('params', {}) or {}
            target_id = params.get('target_template_id')
            if target_id:
                subprocess_refs[node['id']] = {
                    'target_template_id': target_id,
                    'target_version': target.version or 1,
                    'target_name': target.name,
                    'variable_mapping': params.get('variable_mapping', {}),
                    'output_mapping': params.get('output_mapping', {}),
                }
    # 冻结当前 pipeline_tree → snapshot 字段（含子流程引用）
    self.snapshot = {
        'pipeline_tree': self.pipeline_tree,
        'target_hosts': self.target_hosts,
        'global_vars': self.global_vars,
        'subprocess_refs': subprocess_refs,
        'snapshot_at': timezone.now().isoformat(),
    }
    # 创建版本历史记录（含版本说明）
    TemplateVersion.objects.create(
        template=self, version=self.version,
        pipeline_tree=self.pipeline_tree,
        target_hosts=self.target_hosts,
        global_vars=self.global_vars,
        version_note=version_note,
        created_by=created_by if user is None else user,
    )
    self.version += 1
    self.save()
```

****关键设计:**** [snapshot] 是发布时的深拷贝，后续模板修改不影响已发布版本。

第二阶段：创建执行

用户在“执行列表”页面创建新执行：

```
POST /api/opsflow/executions/
{
    "template": 1
}
```

2.1 冻结快照

****文件:**** [views/execution_views.py:perform_create()]（第 28-45 行）

```
def perform_create(self, serializer):
    execution = serializer.save(created_by=self.request.user)
    template = execution.template

    # 优先用发布快照(snapshot), 降级用当前 pipeline_tree
    snapshot_tree = template.snapshot.get('pipeline_tree') if template.snapshot else None
    if not snapshot_tree:
        snapshot_tree = template.pipeline_tree

    # 冻结到 template_snapshot 字段 (执行隔离)
    execution.template_snapshot = {
        'pipeline_tree': snapshot_tree,
        'target_hosts': template.target_hosts,
        'global_vars': template.global_vars,
        'template_version': template.version,
    }
    execution.save()
```

2.2 FlowExecution 表记录

此时 [ops_flow_execution] 表状态：

字段	示例值	说明
`id`	42	执行 ID
`template_id`	1	关联模板 ID
`status`	`pending`	初始状态
`template_snapshot`	`{pipeline_tree: ..., target_hosts: ...}`	**冻结的快照**
`context`	`{}`	运行时上下文 (bamboo_pipeline_id 执行时写入)
`node_status`	`{}`	各节点状态
`state_tree`	`{}`	状态树快照
`schedule_plan_id`	null	手动创建，调度触发时有值
`created_at`	`2026-06-01 09:00:00`	创建时间

第三阶段：启动执行

用户在已创建执行上点击“启动”：

POST /api/opsflow/executions/42/start/

文件： [views/execution_views.py] → [core/flow_engine.py:start()]

3.1 FlowEngine.start()

```
def start(self):
    self.execution.status = "running"          # pending → running
    self.execution.started_at = datetime.now()
    self.execution.node_status = {}
    self.execution.save()

    # 异步派发到 Celery
    from opsflow.tasks import execute_pipeline_task
    execute_pipeline_task.delay(self.execution.id)  # ← 非阻塞
```

****关键:**** `[start()]` 立即返回，实际执行在 Celery worker 中异步进行。

3.2 Celery Worker

****文件:**** `[tasks.py]` (第 47-59 行)

```
@shared_task(bind=True, max_retries=3, queue='er_execute')
def execute_pipeline_task(self, execution_id):
    execution = FlowExecution.objects.get(id=execution_id)
    engine = FlowEngine(execution)
    engine.run()                # ← 核心执行入口
```

第四阶段：引擎运行

4.1 FlowEngine.run()

****文件:**** `[core/flow_engine.py:run()]` (第 134-191 行)

```

def run(self):
    # 1. 从冻结快照读取（不读实时模板！）
    frozen = self.execution.template_snapshot or {}
    frozen_tree = frozen.get('pipeline_tree') or self.execution.context.get('pipeline_tree')

    # 2. 执行时校验 pipeline_tree（防止外部篡改/损坏）
    if frozen_tree:
        validation = validate_pipeline(frozen_tree)
        if not validation.get('valid'):
            errors = '; '.join(validation.get('errors', []))
            self.execution.status = "failed"
            self.execution.ended_at = datetime.now()
            self.execution.save()
            self._notify_completed()
            return

    # 3. 编译成 bamboo-engine 标准 Pipeline Tree（含 excluded_nodes 过滤）
    pipeline = build_bamboo_pipeline(
        self.template,
        pipeline_tree=frozen_tree,
        target_hosts=frozen.get('target_hosts'),
        global_vars=frozen.get('global_vars'),
        execution_id=self.execution.id,
        excluded_nodes=self.execution.excluded_nodes,
    )

    # 空 pipeline 检测
    if len(pipeline.get('activities', {})) == 0 and len(pipeline.get('gateways', {})) == 0:
        self.execution.status = "failed"
        self.execution.ended_at = datetime.now()
        self._notify_completed()
        return

    # 4. 持久化 bamboo_pipeline_id 到 context
    self.execution.context["bamboo_pipeline_id"] = pipeline["id"]
    self.execution.context["bamboo_pipeline"] = pipeline
    self.execution.save()

    # 4b. 创建自动重试策略和超时配置
    AutoRetryStrategyCreator(self.execution, pipeline_id).batch_create_strategy(frozen_tree or {})
    batch_create_timeout_configs(self.execution, frozen_tree or {})

    # 4c. 清理重试时可能残留的 pipeline data
    self._cleanup_pipeline_data(pipeline)

    # 5. 委托给 bamboo-engine 执行（异步）
    runtime = BambooDjangoRuntime()
    result = pipeline_api.run_pipeline(runtime=runtime, pipeline=pipeline)

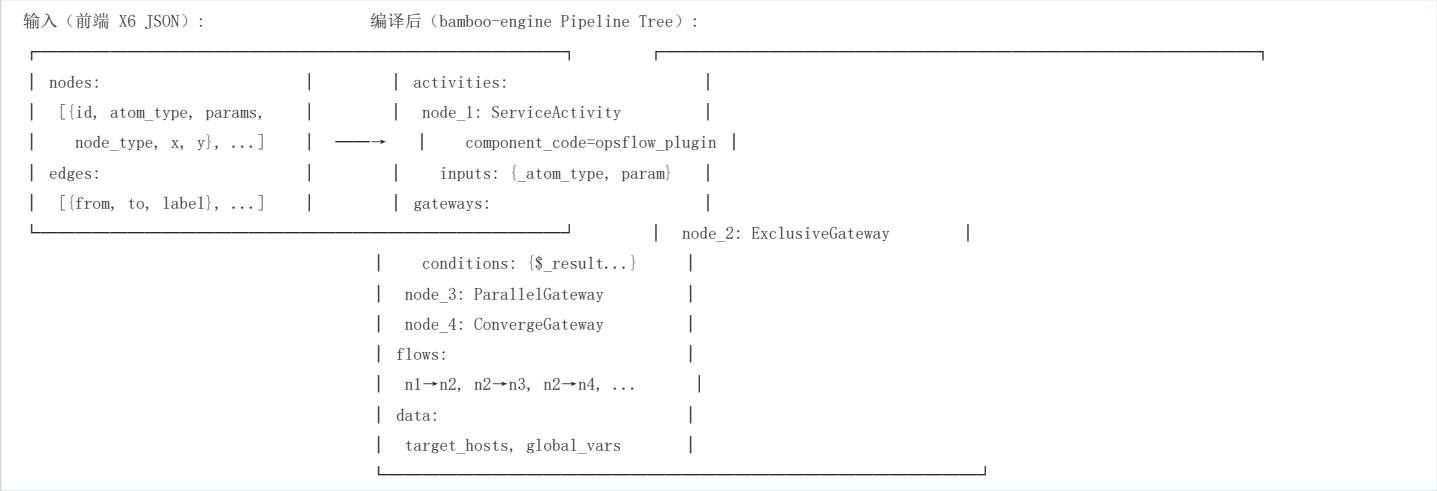
    if not result.result:
        # 启动失败 → 记录校验详情 → 回滚已失败的节点
        self.execution.status = "failed"
        self.execution.ended_at = datetime.now()
        self.rollback_failed_nodes()
        self._notify_completed()
        return

```

4.2 BambooBuilder 编译

****文件:**** [core/bamboo_builder.py:build_bamboo_pipeline()]

将前端 X6 格式的 [pipeline_tree] 编译为 bamboo-engine 可执行的 Pipeline Tree:



节点类型映射规则:

node_type	bamboo-engine 元素	说明
`start_event`	EmptyStartEvent	视觉节点, 不生成
`end_event`	EmptyEndEvent	自动补全
空 / `atom`	ServiceActivity	`component_code`="opsflow_plugin"
`exclusive_gateway`	ExclusiveGateway	条件 `\$_result == True`
`parallel_gateway`	ParallelGateway	扇出所有分支
`conditional_parallel_gateway`	ConditionalParallelGateway	条件扇出
`converge_gateway`	ConvergeGateway	并行汇聚

每个 ServiceActivity 注入:

```
act.component.inputs['_atom_type'] = node.get('atom_type') # 插件路由 ID
act.component.inputs['command'] = params.get('command') # 插件参数
act.component.inputs['timeout'] = params.get('timeout')
```

4.3 BambooEngine 内部执行

bamboo-engine 收到 Pipeline Tree 后, 在 Celery worker 内执行 [Engine.execute()] 循环:

```
Engine.execute(process_id, node_id)
├─ while True:
│   current_node = get_node(current_node_id)
│   handler = get_handler(current_node)
│   result = handler.execute(...)
│   │ post_set_state 信号 → signals.py (每次状态变更都触发)
│   │
│   if result.should_sleep:
│       return ← 睡眠等待 (并行分支/回调)
│   current_node_id = result.next_node_id ← 继续下一节点
```

关键: 这是一个同步 [while] 循环遍历 DAG, 非异步回调。只有遇到并行分支 (ParallelGateway) 或外部回调时才会睡眠。

第五阶段：节点执行（PluginService）

5.1 PluginService 路由

bamboo-engine 调度到 ServiceActivity 节点时，通过 [component_code="opsflow_plugin"] 查找到 [OpsflowPluginComponent] → [PluginService]。

****文件： **** [core/plugin_service_adapter.py]

```
class PluginService(Service):
    def execute(self, data, parent_data):
        inputs = dict(data.inputs)
        atom_type = inputs.pop('_atom_type', '') # 提取插件类型

        # 步骤 1: 从注册表查找 BasePlugin 子类
        plugin_cls = get_plugin(atom_type) # → PLUGIN_REGISTRY

        # 步骤 2: 变量解析 (替换 ${key} 引用)
        pd = dict(parent_data.inputs) if parent_data else {}
        global_vars = pd.get('global_vars', {})
        target_hosts = pd.get('target_hosts', [])
        resolve_ctx = {**global_vars, 'target_hosts': target_hosts}
        resolved_params = resolve_params(params, resolve_ctx)

        # 步骤 3: 实例化 BasePlugin 子类, 执行
        instance = plugin_cls()
        result = instance.execute(**resolved_params)
        success = result.get('success', True)

        # 步骤 4: 写回 outputs → 供下游节点和信号处理器读取
        data.outputs['_result'] = success
        data.outputs['stdout'] = result.get('data', {}).get('stdout', '')
        data.outputs['stderr'] = result.get('data', {}).get('stderr', '')
        data.outputs['executor_output'] = result.get('data', {})

        return success
```

5.2 执行示例

```
atom_type = "shell"
params = {"command": "df -h", "timeout": 60}
resolve_ctx = {"env": "prod", "target_hosts": ["192.168.1.100"]}
```

→ PluginService.execute()

→ get_plugin("shell") → ShellPlugin (plugins/ansible/shell.py)

→ resolve_params({"command": "df -h"}, resolve_ctx) # 无 \${} 引用, 无需替换

→ ShellPlugin().execute(command="df -h", timeout=60)

→ subprocess.run(["df", "-h"], timeout=60)

→ 返回 {"success": True, "data": {"stdout": "Filesystem...", ...}}

5.3 调度/长任务模式

对于长任务（如 Ansible Tower 作业），PluginService 支持 [schedule()] 回调：


```
# BasePlugin 子类设置
class LongTaskPlugin(BasePlugin):
    _need_schedule = True      # 启用异步调度模式

    def execute(self, **kwargs):
        # 触发远程作业, 返回 job_id
        job_id = tower_service.launch_job(...)
        return {"success": True, "data": {"job_id": job_id}}

    def schedule(self, context, **kwargs):
        # bamboo-engine 定期调用此方法 (每秒)
        job_id = context.get('executor_output', {}).get('job_id')
        status = tower_service.poll_job(job_id)
        if status == 'successful':
            return True      # 作业完成
        elif status == 'failed':
            return False     # 作业失败
        return None         # 继续等待
```

第六阶段：状态追踪（信号驱动）

bamboo-engine 每次状态变更触发 [post_set_state] Django 信号。

****文件:**** [signals.py] (第 28-66 行)

```
post_set_state 信号
|
|— 节点 == 根节点 (总 pipeline 状态)
|   └─ _handle_root_state_change()
|       bamboo 状态 → 映射 → OpsFlow 状态
|       FINISHED → completed
|       FAILED → failed
|       REVOKED → cancelled
|       SUSPENDED → paused
|       RUNNING → running
|
|— 节点 != 根节点 (子节点状态)
|   └─ _update_state_tree() → FlowExecution.state_tree 字段
|   └─ _record_node_trace() → NodeExecutionTrace 表 (轨迹记录)
|   |
|   └─ RUNNING → execution.current_node = node_id → WebSocket 推送
|   |
|   └─ FINISHED/FAILED →
|       └─ _log_node_result() → OpsLog 表 (审计日志)
|       └─ _write_node_trace_log() → 写入轨迹完成状态
|       └─ _notify_node_status() → WebSocket 推送
```

状态映射表

bamboo-engine 状态	OpsFlow 状态	说明
`CREATED`	`pending`	初始
`RUNNING`	`running`	执行中
`FINISHED`	`completed`	正常完成
`FAILED`	`failed`	异常终止
`REVOKED`	`cancelled`	用户取消
`SUSPENDED`	`paused`	用户暂停

bamboo-engine 状态	OpsFlow 状态	说明
`BLOCKED`	`failed`	阻塞（子流程失败）

节点执行轨迹

每次子节点状态变更都会记录到 [NodeExecutionTrace] 表，用于前端执行详情页的时序图展示：

字段	说明
`node_id`	节点 ID
`node_label`	节点标签
`status`	状态（running/completed/failed）
`retry_count`	重试次数
`duration_ms`	执行耗时（ms）
`entered_at`	进入时间
`exited_at`	退出时间
`outputs`	节点输出（仅终态记录）
`error`	错误信息

第七阶段：执行控制

7.1 操作与对应处理

操作	用户操作	后端处理	bamboo-engine API
完成	自动	信号 FINISHED → status=completed	—
失败	自动	信号 FAILED → status=failed + rollback	—
取消	点 Cancel	`engine.cancel()`	`revoke_pipeline()`
暂停	点 Pause	`engine.pause()`	`pause_pipeline()`
恢复	点 Resume	`engine.resume()`	`resume_pipeline()`
重试	点 Retry	`engine.retry(node_id)`	`retry_node()`
跳过	点 Skip	`engine.skip(node_id)`	`skip_node()`

7.2 失败回滚

当 pipeline 失败时，[FlowEngine.rollback_failed_nodes()] 遍历所有失败节点，调用各插件的 [BasePlugin.rollback()]：

```
def rollback_failed_nodes(self):
    failed_nodes = [...]
    for node_id in failed_nodes:
        outputs = pipeline_api.get_execution_data_outputs(runtime, node_id)
        _trigger_plugin_rollback(node_id, outputs)
    # → PluginService.rollback()
    # → BasePlugin.rollback(context=outputs, **params)
```

第八阶段：并行/条件分支

8.1 并行分支

bamboo-engine 内置 fork/join 机制：

ParallelGateway → fork:

```
├─ child_1 → [分支 A 节点] → ConvergeGateway
├─ child_2 → [分支 B 节点] → ConvergeGateway
└─ parent 睡眠
```

子进程独立执行：

```
child_1: while True → 节点执行 → should_sleep → ConvergeGateway
child_2: while True → 节点执行 → should_sleep → ConvergeGateway
```

每个子进程到达 ConvergeGateway:

```
→ ack_num++ (原子递增, SELECT FOR UPDATE)
→ ack_num == need_ack → 唤醒父进程 → 继续后续节点
```

8.2 条件分支

ExclusiveGateway 评估条件：

```
${_result == True} → success 分支 (节点执行成功)
${_result == False} → failure 分支 (节点执行失败)
```

由 bamboo_builder.py 在编译时写入条件表达式。

完整流程图示

```

| 前端 DesignCanvas |
| ① 拖拽节点 → 配置参数 → 保存 → PATCH /api/opsflow/templates/{id}/ |
| ② 点击发布 → POST /publish/ → 冻结 pipeline_tree + 版本号+1 |
| ③ 新建执行 → POST /executions/ → 冻结 template_snapshot |
| ④ 启动 → POST /start/ → status=running, Celery 派发 |

```



```

| Celery Worker (queue: er_execute) |
| | | |
| execute_pipeline_task(execution_id) |
| ↓ |
| FlowEngine.run() |
| └─ 1. 读 template_snapshot (冻结的 pipeline_tree, 非实时模板) |
| └─ 2. validate_pipeline() 安全校验 |
| └─ 3. build_bamboo_pipeline(frozen_tree) 编译 |
| | └─ 节点 → ServiceActivity / ExclusiveGateway / ParallelGateway |
| | └─ 边 → 拓扑连接 |
| | └─ 注入 _atom_type → 每个 ServiceActivity 都带插件标识 |
| └─ 4. 持久化 bamboo_pipeline_id → context |
| └─ 5. pipeline_api.run_pipeline() → 异步调度 |
| | | |
| bamboo-engine Engine.execute() 循环: |
| while True: |
|     node = get_node(current_node_id) |
|     handler = get_handler(node) |
|     execute_result = handler.execute() |
|     | |
|     | post_set_state 信号 → signals.py (每次状态变更) |
|     | |
|     if execute_result.should_sleep: return ← 并行/回调时睡眠 |
|     current_node_id = execute_result.next_node_id ← 继续 DAG 遍历 |
| | | |
| PluginService.execute() → 路由到 BasePlugin: |
| └─ get_plugin("shell") → ShellPlugin.execute(command="df -h") |
| └─ get_plugin("esxi_power_on") → EsxiPowerOnPlugin.execute(...) |
| └─ get_plugin("netapp_create_volume") → NetAppCreateVolume.execute(...) |
| └─ ... |

```



```

| signals.py (post_set_state 信号处理器) |
| | | |
| root_state_change → FlowExecution.status |
| FINISHED → completed | FAILED → failed | |
| SUSPENDED → paused | RUNNING → running |
| | | |
| child_state_change → |
| └─ _update_state_tree() → state_tree JSON field |
| └─ _record_node_trace() → NodeExecutionTrace table |
| └─ FINISHED/FAILED → _log_node_result() → OpsLog table |
| └─ FINISHED/FAILED → _notify_node_status() → WebSocket 推送 |
| └─ RUNNING → _notify_node_status() → WebSocket 推送 |
| | | |
| 完成 → WebSocket execution.completed |
| 失败 → WebSocket execution.failed + rollback_failed_nodes() |

```

关键设计原则

原则	实现	文件
执行隔离	template_snapshot 在创建时冻结，引擎始终读快照	`execution_views.py:perform_create()` + `flow_engine.py:run()`
版本追溯	每次发布存 TemplateVersion，支持回滚	`models.py:publish_snapshot()`
异步驱动	start() 立即返回，Celery 异步执行	`flow_engine.py:start()` → `tasks.py:execute_pipeline_task()`
状态信号驱动	bamboo-engine 状态变更 → post_set_state 信号 → 订阅者更新	`signals.py:on_post_set_state()`
统一插件路由	所有原子通过 PluginService + PLUGIN_REGISTRY 运行时查找	`plugin_service_adapter.py:PluginService.execute()`
平台无关	BasePlugin 子类实现特定平台逻辑，execute() 统一接口	`plugins/base.py` + 各 `plugins/{group}/*.py`

数据隔离示意图

